

Міністерство освіти і науки України

Національний технічний університет України

«Київський політехнічний інститут імені Ігоря
Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

Лабораторна робота №6

**з дисципліни «Об'єктно-орієнтоване
програмування»**

на тему

**«Програмна система з компонентами для обробки
даних»**

Виконав:

Степаненко Денис
студент групи ІМ-051
номер у списку групи: 16

Перевірів:

Рекечинський Дмитро Олександрович

Київ 2026

Завдання

1. Створити у середовищі MS Visual Studio C++ проєкт Win32 з ім'ям Lab6.
2. Написати вихідний текст програми згідно варіанту завдання.
3. Скомпілювати вихідний текст і отримати виконуваний файл програми.
4. Перевірити роботу програми. Налогодити програму.
5. Проаналізувати та прокоментувати результати та вихідний текст програми.

Завдання згідно варіанту

Номер у списку: **Ж = 16, варіант = Ж mod 4 = 0**

Три незалежні програми, що обмінюються даними через `WM_COPYDATA` та Clipboard:

- **Lab6 (Менеджер)** — користувач вводить параметри: `nPoint`, `xMin`, `xMax`, `yMin`, `yMax`. Менеджер запускає `Object2` та `Object3` через `WinExec`, розташовує вікна на екрані, передає параметри через `WM_COPYDATA`.
- **Object2 (Генератор даних)** — генерує `nPoint` випадкових пар (x, y) у заданих діапазонах, відображає значення у вікні, записує дані у Clipboard (`CF_TEXT`, `tab-separated`), повідомляє `Object3` через `PostMessage(WM_USER+100)`.
- **Object3 (Побудова графіка)** — читає дані з Clipboard, сортує за x , малює графік $y=f(x)$ з осями координат та підписами.

Вихідний текст програми

Програма `Lab6.cpp` (Менеджер)

```

struct Lab6Params {
    int nPoint;
    int xMin, xMax;
    int yMin, yMax;
    HWND hWndSender;
};

static void LaunchPrograms(HWND hWnd)
{
    // Build paths to Object2.exe and Object3.exe
    TCHAR path2[MAX_PATH], path3[MAX_PATH];
    GetModuleFileName(NULL, path2, MAX_PATH);
    TCHAR* p = _tcsrchr(path2, _T('\\'));
    if (p) *p = 0;
    _stprintf_s(path3, MAX_PATH, _T("%s\\Object2.exe"), path2);
    // ... similar for Object3

    // Check if already running
    hObj2 = FindWindow(_T("Object2Class"), NULL);
    hObj3 = FindWindow(_T("Object3Class"), NULL);

    if (!hObj2) WinExec(cmd2, SW_SHOWNORMAL);
    if (!hObj3) WinExec(cmd3, SW_SHOWNORMAL);

    // Wait for windows to appear
    for (int i = 0; i < 50; i++)
    {
        if (!hObj2) hObj2 = FindWindow(_T("Object2Class"), NULL);
        if (!hObj3) hObj3 = FindWindow(_T("Object3Class"), NULL);
        if (hObj2 && hObj3) break;
        Sleep(100);
    }

    // Position windows on screen
    MoveWindow(hObj2, 520, 50, 400, 300, TRUE);
    MoveWindow(hObj3, 520, 370, 400, 400, TRUE);
    MoveWindow(hWnd, 50, 50, 450, 350, TRUE);
}

static void SendParams(HWND hWnd)
{
    Lab6Params params;
    // Read from edit controls
    params.nPoint = _ttoi(buf); // etc.

    // Send to Object2 via WM_COPYDATA
    if (hObj2)
    {
        COPYDATASTRUCT cds;
        cds.dwData = 0;
        cds.cbData = sizeof(Lab6Params);
    }
}

```

```

        cds.lpData = &params;
        SendMessage(hObj2, WM_COPYDATA, (WPARAM)hWnd, (LPARAM)&cds);
    }
}

```

Програма Object2.cpp (Генератор даних)

```

static void GenerateData()
{
    g_count = g_params.nPoint;
    g_x = new double[g_count];
    g_y = new double[g_count];

    srand((unsigned)time(NULL));
    for (int i = 0; i < g_count; i++)
    {
        double rx = (double)rand() / RAND_MAX;
        double ry = (double)rand() / RAND_MAX;
        g_x[i] = g_params.xMin + rx * (g_params.xMax - g_params.xMin);
        g_y[i] = g_params.yMin + ry * (g_params.yMax - g_params.yMin);
    }
    InvalidateRect(g_hWnd, NULL, TRUE);
}

static void WriteToClipboard()
{
    // Format: "x\ty\n" per line
    char* buf = new char[g_count * 32];
    int pos = 0;
    for (int i = 0; i < g_count; i++)
        pos += sprintf_s(buf + pos, ..., "%.2f\t%.2f\n", g_x[i], g_y[i]);

    HGLOBAL hglb = GlobalAlloc(GMEM_MOVEABLE, pos + 1);
    memcpy(GlobalLock(hglb), buf, pos + 1);
    GlobalUnlock(hglb);
    OpenClipboard(g_hWnd);
    EmptyClipboard();
    SetClipboardData(CF_TEXT, hglb);
    CloseClipboard();

    // Notify Object3
    HWND hObj3 = FindWindow(_T("Object3Class"), NULL);
    if (hObj3)
        PostMessage(hObj3, WM_USER + 100, 0, 0);
}

// Handler:
case WM_COPYDATA:
{
    COPYDATASTRUCT* cds = (COPYDATASTRUCT*)lParam;
    if (cds && cds->cbData == sizeof(Lab6Params))

```

```

{
    memcpy(&g_params, cds->lpData, sizeof(Lab6Params));
    GenerateData();
    WriteToClipboard();
}
}

```

Програма Object3.cpp (Побудова графіка)

```

#define WM_CLIPBOARD_NOTIFY (WM_USER + 100)

static void ReadFromClipboard()
{
    OpenClipboard(g_hWnd);
    HGLOBAL hglb = GetClipboardData(CF_TEXT);
    char* text = (char*)GlobalLock(hglb);

    // Parse "x\ty\n" lines
    int lines = 0;
    // ... count lines, allocate arrays, parse values

    GlobalUnlock(hglb);
    CloseClipboard();

    // Sort by x (ascending)
    for (int i = 0; i < g_count - 1; i++)
        for (int j = i + 1; j < g_count; j++)
            if (g_x[j] < g_x[i]) { /* swap */ }

    InvalidateRect(g_hWnd, NULL, TRUE);
}

// In WM_PAINT: draw axes + graph line
case WM_PAINT:
{
    // Find min/max ranges
    // Draw X axis (bottom), Y axis (left)
    // Draw axis labels (xMin, xMax, yMin, yMax)
    // Draw graph: red line connecting sorted points
    for (int i = 0; i < g_count; i++)
    {
        int px = m1 + (int)((g_x[i] - xMin) / (xMax - xMin) * gw);
        int py = mt + gh - (int)((g_y[i] - yMin) / (yMax - yMin) * gh);
        if (i == 0) MoveToEx(hdc, px, py, NULL);
        else LineTo(hdc, px, py);
    }
}
}

```

Діаграми

Діаграма залежностей файлів та модулів

```
Lab6/
├─ Lab6.cpp      ← (standalone, only <windows.h>)
├─ Lab6.rc      ← dialog resource

Object2/
├─ Object2.cpp   ← (standalone, only <windows.h>)
└─ (no .rc needed – simple window)

Object3/
├─ Object3.cpp   ← (standalone, only <windows.h>)
└─ (no .rc needed – simple window)
```

Усі три програми — **повністю незалежні**. Жодних спільних `.h` файлів. Зв'язок між ними — лише через `WM_COPYDATA` , `Clipboard` та `PostMessage` .

Діаграма компонентів системи

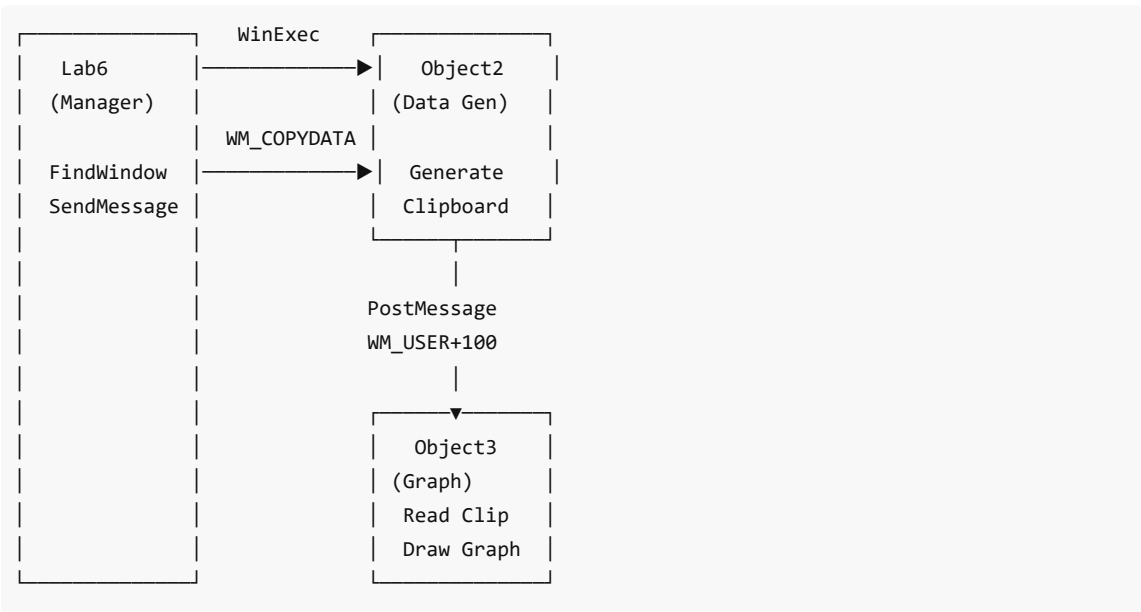
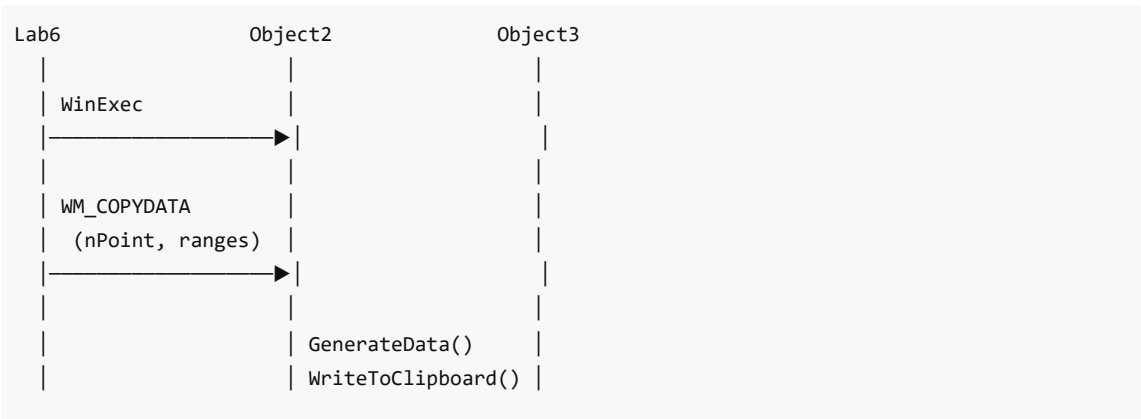
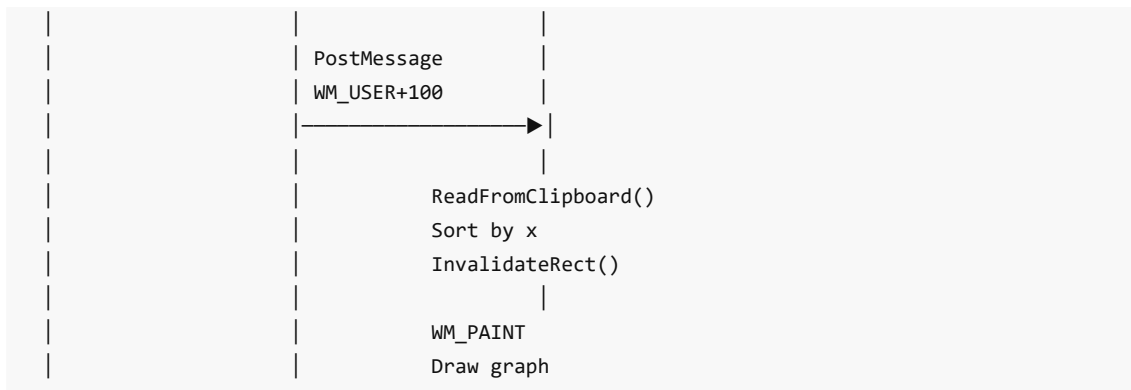


Схема послідовності обміну повідомленнями





Скріншоти

Вікно менеджера Lab6



Рис. 1. Вікно менеджера Lab6 — ввід параметрів $nPoint$, $xMin$, $xMax$, $yMin$, $yMax$

Три вікна одночасно



Рис. 2. Три вікна одночасно: менеджер (зліва), Object2 (справа вгорі), Object3 (справа внизу)

Object2 — згенеровані дані



Рис. 3. Програма Object2 — список згенерованих пар (x, y)

Object3 — графік $y=f(x)$



Рис. 4. Програма Object3 — графік $y=f(x)$ з осями координат та підписами

Висновки

У лабораторній роботі я виконав завдання згідно свого варіанту ($\mathcal{K} = 16$, варіант 0). Створено програмну систему з трьох незалежних програм, що обмінюються даними через механізми Windows API.

Lab6 (Менеджер) запускає Object2 та Object3 через `WinExec`, знаходить їхні вікна через `FindWindow` за ім'ям класу, передає параметри через `WM_COPYDATA` зі структурою `COPYDATASTRUCT`, та розташовує вікна на екрані через `MoveWindow`.

Object2 (Генератор) приймає параметри через `WM_COPYDATA`, генерує випадкові пари (x, y) у заданих діапазонах, записує їх у Clipboard у форматі `CF_TEXT` (tab-separated), та повідомляє Object3 через

`PostMessage(WM_USER+100)` .

Object3 (Графік) отримує повідомлення-notify, читає дані з Clipboard, сортує за x , малює графік $y=f(x)$ з осями координат, підписами меж та лінією графіка.

Лабораторна робота дала практичні навички розробки багатопрограмних систем, використання

`WM_COPYDATA` для передачі даних між процесами, роботи з Clipboard (`OpenClipboard` , `SetClipboardData` , `GetClipboardData`), та координації вікон через `FindWindow` і `PostMessage` .